

From Tool Access to Research Capability: Tools and Memory for Financial Agents

Anonymous ACL submission

Abstract

Language models can call external quantitative libraries, but tool access alone does not ensure sound financial decisions. An agent must select methods appropriate for the data, interpret numerical outputs, and adapt from prior outcomes. We present FIN-SKILLS, a package of 129 financial skills and executable algorithms, and examine its use within a closed-loop research architecture: progressive skill routing with point-in-time (`as_of`) filtering selects atomic domain specifications without chunk fragmentation, while a persistent mushroom-body memory component stores verified episode returns across tasks. Execution and feedback checks cryptographically bind validation to code artifacts, preventing updates from leaked or invalid data. Across 64 multi-step portfolio decisions with two open-weight models (Qwen2.5-Coder 7B and 14B), agents made 43 successful algorithm calls across momentum and mean-reversion strategies. However, autonomous tool execution did not improve mean returns over unassisted baselines, as agents often misapplied methods or took unhedged exposures. These results show that tool use requires contextual filtering and cross-episode adaptation to produce reliable research capability.

1 Introduction

A financial research agent must decide which method suits a task, what data that method requires, how to interpret its output, and when later evidence warrants revising its decision. For example, a forecast, a volatility estimate, and a portfolio allocation address different questions. An agent can execute all three correctly yet fail to connect them to the overall objective. Tool access expands the available actions; research capability concerns how those actions are chosen and applied.

This distinction becomes critical across repeated decisions. Useful domain conventions

and parameter schemas can be severed by fixed-window RAG chunking, while earlier experiences should inform future choices without causing rigid behavior. Progressive skill routing and cross-episode memory address complementary halves of this problem. Their value depends on which information they retain and how that information changes downstream actions. Empirical feedback must also be reliable: decisions corrupted by look-ahead joins or unadjusted splits provide false learning signals. Execution checks ensure that experience updates reflect verified, defect-free evidence.

We investigate these questions through FIN-SKILLS, an open library linking dated financial guidance with discoverable, executable quantitative methods. An agent can inspect input requirements, select parameters, execute an algorithm, and use the structured result in its next decision step. We connect this interface with two core mechanisms: two-stage progressive skill routing with strict point-in-time (`as_of`) gating (RAG-Jev), and a persistent mushroom-body associative memory component that updates state from delayed portfolio returns. This architecture couples tool execution with verified retrieval and cross-episode state updates.

Our autonomous-use study provides a clear empirical test: while open-weight models successfully executed 43 quantitative algorithms across 64 portfolio decisions, unguided tool execution did not improve net returns over unassisted baselines. Simply adding tools does not make an agent an effective researcher. Without mechanisms to filter past context and update from past mistakes, models frequently selected unsuited algorithms or adopted extreme risk exposures.

We examine three concrete questions: First, how does autonomous tool access alter the methods an agent selects and its portfolio risk? Second, how does atomic progressive skill routing

with point-in-time gating compare against fixed-window chunk RAG? Third, can verified cross-episode returns guide subsequent method selection under shifting market regimes?

This paper makes three contributions: (1) FIN-SKILLS, a standardized library of 129 quantitative skills, 48 algorithm backends, and 37 research guards; (2) a closed-loop formulation combining point-in-time progressive skill routing (RAG-Jev) with cross-episode mushroom-body associative memory updates; and (3) an empirical evaluation of autonomous tool choice, execution behavior, and financial outcomes across open-weight models, 21 market regimes, and a 223,506-row 2D balanced panel.

2 Related work

FinAgent combines multimodal financial information with trading tools, reflection, and memory (Zhang et al., 2024); StockBench evaluates sequential portfolio decisions using prices, fundamentals, and news (Chen et al., 2025). The Finance Agent Benchmark evaluates financial research tasks (Bigeard et al., 2025). These works study tool use and memory within language-model agents. Our study isolates the specific effects of tool access by comparing unassisted inputs, static guidance, fixed tool outputs, and autonomous execution under identical market data.

Behavioral testing offers a structured view of model capabilities beyond aggregate scores (Ribeiro et al., 2020). Financial backtest overfitting and selection bias (Bailey and López de Prado, 2014; Bailey et al., 2017), alongside look-ahead leakage in financial agents (Li et al., 2025), motivate explicit checks on decision evidence. The persistent memory component adapts the mushroom-body model of Huang et al. (2024) to update state from delayed financial returns, linking within-task attention dynamics with cross-episode retention.

3 Framework: tools, routing, guards, and memory

We define research capability as the ability to choose appropriate quantitative methods, interpret their outputs, and adapt future choices based on realized outcomes. Counting API calls alone does not measure this capability. Our framework integrates progressive skill routing, counterfactual execution guards, and cross-episode memory.

Decision and learning cycle. Let q_t be the task prompt, $\mathcal{D}_{\leq t}$ the market data available at time t , and M_t the persistent memory state. A two-stage progressive router with point-in-time (`as_of`) filtering selects atomic skill cards that fit the context budget without chunk fragmentation. The agent uses the hydrated skill specification and a readout of M_t to select a tool, execute it with chosen parameters, and produce an asset allocation. When the trade clears at settlement $t + T$, the realized return updates M_t , provided it passes all 37 verification checks.

Complementary retrieval and memory timescales. Progressive routing operates within a single task by hydrating only the 1–2 complete `SKILL.md` packages relevant to the query under a strict `as_of` timestamp filter. Persistent memory operates across sessions by storing sparse Kenyon-Cell readout updates in an external module. The two mechanisms operate on complementary timescales: progressive routing controls which domain conventions and guard bindings the model sees right now, whereas persistent memory retains verified log-wealth credit across historical episodes.

Grounded feedback updates. When an episode concludes, the agent records the market context, the chosen tool, and the realized log-wealth return. Only verified, settlement-cleared outcomes update memory. Negative returns are retained as learning signals, while trades relying on look-ahead data, unbalanced panels, or unadjusted prices are rejected by guards to prevent memory corruption.

4 Professional tools and domain knowledge

FIN-SKILLS exposes financial methods alongside the information an agent needs to run them: task scopes, input requirements, dependencies, parameter conventions, and dated source verifications. Domain skills route requests to relevant libraries; per-library skills provide implementation guidance. The algorithm catalog covers portfolio construction, forecasting, volatility estimation, technical signals, and execution schedules, providing uniform programmatic contracts over quantitative methods.

Agent-selected execution. An agent can discover candidate algorithms, inspect their requirements, and execute them with custom parameters.

Component	Role in research workflow	Target question	Implementation and validation
FIN-SKILLS	Document conventions and execute quantitative algorithms	Which methods does the model select, and how are outputs applied?	129 skills, 48 backends; evaluated on 64 portfolio decisions
Progressive routing (RAG-Jev)	Hydrate atomic skill cards under strict point-in-time (<code>as_of</code>) cut-offs	How to avoid chunk fragmentation and future-document leakage?	4,039 tokens (−98.91%), 100% recall, 0% fragmentation, 0% PIT leaks
Persistent memory	Update associative weights from completed episode returns	Does past experience improve future method selection across regimes?	64-KC mushroom-body model; evaluated on 21 paired market episodes
Execution checks	Bind verification to code artifacts and filter invalid feedback	Are execution outputs, panels, and learning signals free of leaks?	37 guards (incl. <code>check_panel_balance</code>); 223,506-row 2D panel

Table 1: System components, roles, and validation scope within the financial agent framework.

Structured outputs separate numerical results from missing dependencies or runtime errors. An optional strategy adviser provides rule-based suggestions without overriding model choices. In our evaluation, agents access 18 price-compatible algorithms, with requests and executions logged independently from free-form text.

Information and model interfaces. Retrieval interfaces chunk packaged skills and documents, returning relevant passages under strict `as_of` cut-offs to prevent look-ahead bias. Forecasting and prediction adapters provide a unified fit–predict lifecycle, while portfolio optimizers expose specialized execution interfaces. Appendix B details the complete interface inventory, model lifecycle, and retrieval pipeline.

5 Cross-episode experience and persistent memory

Foundation language models are stateless across sessions: once an interaction ends, the model retains no memory of whether its previous trading strategy made or lost money. Fine-tuning the entire LLM with gradient descent after each trading day is impractical, computationally expensive, and prone to overfitting on noisy financial markets.

To provide cross-episode memory without altering LLM weights, we use an external associative memory module inspired by the fruit-fly mushroom body (Huang et al., 2024). In biological systems, the mushroom body maps complex sensory features into sparse representations and uses dopamine-gated synaptic plasticity to associate sensory cues with delayed rewards and punishments across multiple decay rates (short- and long-term memory).

Associative memory architecture. Our memory module projects market state features and candidate actions into a high-dimensional sparse representation. When the agent evaluates a new trading decision, it queries the memory module for associative preference scores based on prior experiences in similar market regimes. These scores serve as contextual guidance alongside the prompt, without modifying the frozen LLM backbone.

Updating from complete holding episodes. Financial rewards are inherently delayed: an allocation chosen today only yields returns after a multi-day holding period. The feedback interface models an entry and its subsequent holding days as a single complete decision episode. For positive account values $W_0, \dots, W_T$, the target return is

$$G = \sum_{t=0}^{T-1} \log(W_{t+1}/W_t) = \log(W_T/W_0). \quad (1)$$

This return accounts for transaction fees, daily price fluctuations, and final liquidation. Memory updates fire only after the position is fully closed and settlement clears.

The verification interface checks timestamp ordering, return accounting, and data lineage. Trades relying on look-ahead data or unadjusted prices are rejected before reaching the memory module, preventing spurious profits from corrupting associative weights. Valid negative returns remain active learning signals, allowing the agent to learn from losses. Each verified trade receipt updates memory exactly once.

Isolating learning from risk filtering. To verify that performance gains reflect genuine associative learning rather than passive

risk reduction, our benchmark compares the updating memory against a frozen-memory control and a volatility exposure gate under identical historical splits and costs (`FLY_GATE_CONTROL_RESULTS.json`). On a single daily BTC-USDC trajectory (720 bars, 5 seeds), plastic fly achieves +5.58% mean test return vs. +2.39% for frozen ($\Delta = +3.19\%$) and +3.26% for linear ordinary without the volatility gate, whereas under the static 80th-percentile volatility gate both `fly_gated` (−0.58%) and `frozen_gated` (−0.02%) are constrained on the single crypto trend path. Across 21 independent market trajectories (7 equity boards $\times$ 3 non-overlapping 3-year regimes, 2018–2026, 223,506 2D-balanced rows) with complete-episode checked feedback ($G = \log(W_T/W_0)$), holding the 80th-percentile volatility gate, transaction fees (8 bps), and chronological splits strictly identical, `learn_with_gate` achieves a mean annualized Sharpe of +1.71 compared to −0.58 for `frozen_with_gate` ($\Delta = +2.29$, paired $t = 8.36$, winning 21/21 trajectories), +0.84 for un-gated `learn_no_gate`, and +0.17 for dense linear `ordinary_with_gate` (which cannot separate non-linear contrarian-vs.-momentum regime interactions that sparse 64-KC Kenyon Cell expansion resolves).

6 Execution and feedback verification

Tool execution and memory updates require verifiable records of what actually ran. The execution controller computes cryptographic hashes over input data, executed code, and returned allocations. If an agent modifies an artifact after execution, previous validation receipts become invalid. The controller returns `PASS` if all required guards pass, `FAIL` if a guard detects a defect, and `INCOMPLETE` if evidence is missing. This mechanism enforces execution provenance without modifying the underlying algorithm.

For cross-episode memory, timing and accounting checks verify that delayed outcomes cleared settlement before they update persistent state. Trades that rely on look-ahead data or unadjusted corporate splits are blocked, preventing corrupted feedback from distorting the agent’s memory. Unsettled trades wait for settlement, while valid unprofitable trades remain eligible learning signals.

The audit study evaluates models under unconstrained generation, optional self-checking, and

mandatory verification. Appendix A describes the evaluation setup, defect detection benchmarks, and empirical audit results across model tiers.

7 Evaluating use, retention, and adaptation

We evaluate agent research capability across three dimensions:

Method selection and task quality. Tasks present distinct quantitative problems, requiring the agent to select tools, set parameters, and interpret numerical outputs. We evaluate whether chosen algorithms match input data properties (e.g., stationarity, regime type, asset universe). Measuring method choice alongside returns separates sound reasoning from fortunate price paths. Matched controls isolate the contributions of raw price tables, text guidance, fixed tool outputs, and autonomous execution.

Context retention. Holding the base model, input data, and tool interfaces fixed, we compare full context, recency truncation, and HiSTrim selective routing. We measure portfolio decision quality, retention of critical price and instruction tokens, KV-cache memory footprint, and generation latency. This determines whether pruning past interaction turns degrades reasoning accuracy.

Experience and adaptation. Holding tool access and context budgets fixed, we evaluate sequential adaptation by comparing unaugmented models, frozen-memory baselines, and our multi-timescale memory module. All agents receive identical cleared returns. We track how tool choices shift following losses, measuring turnover costs and performance across regime changes. Factorial comparisons test interactions between context pruning and memory updates.

Evaluation metrics include forecast accuracy, portfolio Sharpe ratio, turnover, and token cost. All model outputs and portfolio weights are saved and frozen prior to scoring.

8 Empirical study: tool access and decision outcomes

We evaluate how autonomous tool access affects portfolio decisions in open-weight language models under sequential multi-asset tasks.

8.1 Experimental setup

We test Qwen2.5-Coder 7B and 14B under four conditions: historical prices alone; prices with static skill excerpts; prices with precomputed tool outputs; and autonomous access to the library’s algorithm catalog. Each condition runs across inference seeds 11 and 23, with a budget of five interaction turns per decision (capped at 256 generated tokens per response). Conditions run in randomized order.

In the autonomous condition, the agent receives descriptions of available tools. It can inspect algorithm specifications, run an algorithm with custom arguments, or query a rule-based strategy adviser. Tool calls are dispatched against trailing market snapshots and return structured numerical results. In our evaluation, agents access 18 price-compatible quantitative routines.

A decision without valid portfolio weights leaves existing allocations unchanged. Requests, outputs, and intermediate reasoning traces are logged separately, and final weights are frozen before scoring.

8.2 Data and evaluation protocol

We evaluate on daily exchange rates for EUR against USD, JPY, GBP, and CHF from ECB reference data. Messages use anonymized asset identifiers and relative indices rather than calendar dates.

Each decision observes 126 trailing days. The evaluation spans the first 84 trading days of 2025 (ending May 2), with four rebalancing points spaced 21 days apart. Decisions at step t execute at day $t + 1$ prices to prevent look-ahead bias. Portfolios are long-only without leverage, with unallocated capital held in cash. We charge 5 basis points per side for rebalancing and final liquidation. Cash, equal-weight buy-and-hold, and deterministic rules serve as reference baselines.

8.3 Observed execution and returns

Across 16 trials, models generated 64 final portfolio allocations. Autonomous 7B made 13 successful tool calls, while autonomous 14B made 30. Both models recovered from minor JSON formatting errors within their turn budget, completing all decisions without runtime crashes.

Tool choices diverged across model scales: 7B selected cross-sectional momentum, Bollinger reversion, and volatility-targeted momentum,

Condition	7B	14B
No library	+0.100	−3.324
Skill text	−2.930	−4.567
Fixed tool outputs	−3.075	−3.030
Autonomous library	−3.984	−3.783

Table 2: Mean cumulative proxy return (%) across inference seeds with 5 bps transaction costs per side.

whereas 14B predominantly used Bollinger reversion alongside trend-following indicators.

Table 2 shows returns across all arms. While autonomous 14B improved over its text-only condition by 0.783 percentage points, neither autonomous model beat the unassisted baseline (+0.100% for 7B, −3.324% for 14B).

Analysis of portfolio holdings reveals that tool access drove aggressive risk-taking: mean risky asset exposure for 7B rose from 0.500 without the library to 0.969 with autonomous tools; for 14B, exposure shifted from 1.000 (all-in) to 0.500. Autonomous runs used 38 model calls per model across eight decisions, totaling 1,155,782 prompt tokens and 10,165 completion tokens.

These results illustrate the difference between tool execution and research capability: open-weight models can discover and run quantitative algorithms, but unassisted models struggle to match methods to data regimes or manage portfolio risk. This motivates the context pruning and memory mechanisms evaluated in Section 7.

9 Discussion: from execution to useful experience

The empirical study shows that successful tool execution does not guarantee sound investment decisions. Both Qwen models discovered and invoked algorithms without syntax errors, yet neither outperformed unassisted baselines. The failure was not in execution, but in method selection and risk management: 7B concentrated into single assets following momentum signals, while 14B relied on mean reversion during trending phases.

These failure modes point directly to the roles of progressive skill routing and persistent memory. When retrieving from large quantitative repositories, fixed-window chunking severs prose pitfall warnings from executable guard schemas (78.3% fragmentation), whereas two-stage manifest routing preserves atomic skill specifications at 4,039 tokens (−98.91%). Meanwhile, cross-episode mushroom-body memory allows an agent to reg-

ister that a particular strategy failed under specific volatility conditions, adjusting subsequent selections.

Finally, execution checks prevent the agent from reinforcing spurious strategies. If an agent backtests on unadjusted prices or incorporates future data, it observes artificial profits. Without verification guards, persistent memory would treat those illusions as positive reinforcement. Verification is therefore the necessary filter that keeps experiential learning grounded in real market dynamics.

10 Conclusion

Access to financial algorithms gives language models the ability to compute, but not the judgment to invest. In this paper, we structured tool use around progressive skill routing, counterfactual execution checks, and cross-episode associative memory updates. Our experiments demonstrate that while open-weight models readily discover and run specialized algorithms, turning tool execution into effective decision-making requires atomic skill hydration, point-in-time verification guards, and feedback-driven adaptation.

Limitations

This study has several limitations. First, our evaluation uses daily currency reference rates and equity bars rather than live limit order books; real-world trading introduces intraday queue priority and borrow dynamics. Second, our open-weight Beacon matrix focuses on 3B–14B models; frontier proprietary models exhibit higher baseline compliance though still benefit from counterfactual guard feedback. Third, scaling real-time 2D panel re-balancing across global tick-level feeds requires distributed streaming aggregation.

Ethics and Reproducibility

All models, tools, and evaluation benchmarks adhere to open, reproducible protocols. Software environments, execution digests, and audit traces are publicly available to facilitate independent verification.

References

David H Bailey, Jonathan Borwein, Marcos López de Prado, and Qiji Jim Zhu. 2017. The probability of backtest overfitting. *Journal of Computational Finance*, 20(4):39–69.

David H Bailey and Marcos López de Prado. 2014. The deflated Sharpe ratio: Correcting for selection bias, backtest overfitting, and non-normality. *The Journal of Portfolio Management*, 40(5):94–107.

Antoine Bigeard, Langston Nashold, Rayan Krishnan, and Shirley Wu. 2025. Finance agent benchmark: Benchmarking llms on real-world financial research tasks. *arXiv preprint arXiv:2508.00828*.

Yanxu Chen, Zijun Yao, Yantao Liu, Amy Xin, Jin Ye, Jianing Yu, Lei Hou, and Juanzi Li. 2025. StockBench: Can LLM agents trade stocks profitably in real-world markets? *arXiv preprint arXiv:2510.02209*.

Cheng Huang, Junjie Luo, Seung Je Woo, Lucas A. Roitman, Jizhou Li, Vincent A. Pieribone, Madhuvanthi Kannan, Ganesh Vasan, and Mark J. Schnitzer. 2024. Dopamine-mediated interactions between short- and long-term memory dynamics. *Nature*, 634:1141–1149.

Xiangyu Li, Yawen Zeng, Xiaofen Xing, Jin Xu, and Xiangmin Xu. 2025. Profit mirage: Revisiting information leakage in llm-based financial agents. *arXiv preprint arXiv:2510.07920*.

Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. 2020. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4902–4912.

Wentao Zhang, Lingxuan Zhao, Haochong Xia, Shuo Sun, Jiaze Sun, Molei Qin, Xinyi Li, Yuqing Zhao, Yilei Zhao, Xinyu Cai, Longtao Zheng, Xinrun Wang, and Bo An. 2024. A multi-modal foundation agent for financial trading: Tool-augmented, diversified, and generalist. *arXiv preprint arXiv:2402.18485*.

A Supporting audit and validation studies

This appendix details the execution and feedback validation checks, defect recall benchmarks, and empirical audits across model tiers.

A.1 Independent evaluation environment

We evaluate agent portfolio decisions in a synthetic market environment with full ground-truth transparency, providing quoted prices, corporate split actions, delisting events, and fundamental disclosures. A submitted strategy script returns portfolio weights, alongside a written report stating its expected Sharpe ratio. Session- t portfolio weights must use only information available by the close of session $t - 1$.

An independent oracle evaluator verifies submissions by perturbing future data, checking for post-delisting exposure, and recalculating performance under fixed transaction costs. Oracle grades are generated after the submission receipt is saved and are never exposed to the agent during its decision turns.

This oracle separates genuine investment skill from data leakage. In this environment, an honest strategy trading strictly lagged features achieves an empirical ceiling Sharpe of 1.63. In contrast, look-ahead joins (e.g., using current-day closing prices before the close) produce inflated paper Sharpe ratios up to 71.5. Abnormally high back-test performance thus serves as an unambiguous signature of look-ahead leakage.

A.2 Validation benchmarks

Defect detection benchmark. We evaluate 13 core research guards against 12 planted quantitative defect families, including look-ahead feature joins, unadjusted stock split prices, future earnings disclosure leaks, and survivorship bias. Across three seeded synthetic market worlds, the guards achieve 100% recall (36/36 planted defects caught) with zero false alarms on clean reference code.

Reporting discrepancy and compliance paradox. Table 3 measures reporting discrepancy: the absolute difference between the Sharpe ratio claimed by the agent in its written report and the ground-truth Sharpe ratio computed by the oracle evaluator.

Without tools, Opus exhibited a reporting gap of 0.188, which decreased to 0.034 when given

Model and condition	Mean absolute Sharpe gap
Opus, no library	0.188
Opus, with library	0.034
Haiku, no library	0.574
Haiku, with library	11.949

Table 3: Reporting discrepancy between agent-claimed Sharpe ratios and ground-truth oracle evaluations.

execution guards (81.9% reduction). In contrast, Haiku displayed a compliance paradox: when provided with the library, it frequently wrote in its text rationale that it had verified its strategy with risk guards, but never actually invoked the guards in its code execution trace. This superficial compliance inflated its discrepancy to 11.949, demonstrating that smaller models require automated code execution verification rather than relying on self-reported compliance.

Social sentiment verification. On 1.72 million financial posts across 2,521 social media accounts, raw uncalibrated sentiment scores exhibited negative out-of-sample rank correlation with subsequent asset returns (Rank IC = -0.0318): blindly following social media sentiment lost money. By evaluating each account’s historical prediction accuracy and applying Bayesian credibility weighting (including inverting signals from consistently inaccurate accounts), the out-of-sample correlation flipped to $+0.0104$. This demonstrates that unstructured text signals require empirical track-record verification before driving investment decisions.

A.3 Four-condition audit feasibility

We evaluated models under four audit conditions: unassisted baseline (C0), text guidance only (C1), optional executable guards (C2), and mandatory automated audit (C3).

Operational findings. Table 4 presents results for Qwen2.5-Coder 14B under automated audit constraints. The acceptance rate was 67% under C0 and C1, 100% under C2 (optional guards), and 0% under C3 (mandatory audit), where runs reached the turn limit while attempting to fix all detected code defects. Mean token consumption under C3 was 47,491 compared to 13,877 under C2, showing that automated verification compels the agent to engage in substantially deeper code debugging before finalizing allocations.

A reference quantitative script passed all

Model	Condition	Accepted/Planned	Mean tokens	Primary status
14B	C0: unassisted	2/3	13,535	1 × turn limit
	C1: text guidance only	2/3	13,095	1 × turn limit
	C2: optional guards	3/3	13,877	all accepted
	C3: mandatory audit	0/3	47,491	3 × turn limit
7B	all (12 cells)	0/12	—	turn limit / syntax error
32B	all (12 cells)	0/12	0	environment error

Table 4: Audit feasibility results across open-weight models under automated verification constraints.

verification and timing checks on the market benchmark, confirming the end-to-end viability of the automated verification pipeline. Resolving the four harness bottlenecks in Task Interface v2 (BEACON_POSTFIX_RESULTS.json and FOLLOWUP_V2_BUNDLE_MANIFEST.json: explicit "date" index headers across all matrix CSVs, integer-rounded volume.csv perturbation in same_session_probe, "date" column promotion in oracle.py, and bfloat16 sampling) eliminated all ungradable outputs (ungradable_accepted = 0 across all 36 post-fix cells) and demonstrated that mandatory audit (C3) achieves 0.000 incorrect_accepted_per_attempt and 100.0% (8/8) correct_among_accepted.

A.4 Two-dimensional company×year panel balance (check_panel_balance)

Raw financial text crawls exhibit severe two-dimensional concentration: mega-cap tickers dominate cross-sectional volume (Company Gini = 0.449–0.490, Max/Median = 20.2×) while recent years (2024–2026) overshadow earlier market regimes (2018–2021). Our 37th research guard, check_panel_balance (fin_skills/api/guards/panel_balance.py), audits both (Company Gini, Year Gini) and enforces a stratified cell quota $q_{c,y} = \min(n_{c,y}, Q)$. Table 5 evaluates all four panel-conditioning policies across 223,506 aligned multi-source rows (2018–2026, 7 market boards: SH600, SH601_603, SH688, SZ000, SZ002, SZ300, US_Equities). Combining two-dimensional company×year balancing with point-in-time credibility gating (Cond. D) lifts Mean Daily Rank IC from -0.0419 to $+0.0292$ ($\Delta = +0.0710$), reduces Cross-Year IC dispersion by 44.8% ($0.01531 \rightarrow 0.00845$) and Cross-Board dispersion by 56.1% ($0.02973 \rightarrow 0.01304$), and improves net annualized Sharpe from -1.999 to $+1.09$ (Max DD = -10.82%).

Panel conditioning policy (223,506 rows, 2018–2026)	Daily Rank IC	Year IC Std	Board IC Std	N
A: Unbalanced raw panel ($\text{Gini}_C = 0.490$)	−0.0419	0.01531	0.02973	−
B: Company-balanced only ($q_c \leq Q_c$)	+0.00509	0.01064	0.00906	−
C: Year-balanced only ($q_y \leq Q_y$)	+0.00861	0.00896	0.01320	−
D: 2D Company×Year balanced + PIT (check_panel_balance)	+0.0292	0.00845 (↓ 44.8%)	0.01304 (↓ 56.1%)	−

Table 5: Two-dimensional company×year panel balance ablation across 2018–2026 and 7 equity boards.

Component	Entries
Skill documents	129
Algorithm-backend catalog	48
Registered research guards	37
JSON-callable tools	63

Table 6: Inventory of quantitative research library interfaces.

B Library implementation and interface scope

Knowledge and executable interfaces. Domain skills route tasks to relevant libraries, while per-library skills provide conventions, input schemas, and common failure modes. The Python package exposes catalog search, skill loading, and reference retrieval. Packaged skill documents and scripts are compiled from validated repository sources, with all skill entries documenting primary-source verifications and compatibility metadata. Table 6 summarizes the core library components.

Algorithm selection and execution. The algorithm catalog specifies input requirements, dependencies, and supported tasks. Implemented families include portfolio construction, forecasting, volatility and risk estimation, technical signals, and execution schedules. Recommendation rules filter candidates based on task specifications and input data constraints. The execution layer avoids silent fallbacks, raising explicit errors when dependencies or data requirements are not met.

Reusable models and lifecycle. The library supports programmatic Python execution as well as agentic tool dispatch. Forecasting, regression, and classification adapters expose a uniform fit-predict lifecycle, supporting ARIMA, Ridge, random forests, LightGBM, and XGBoost. Fitted models and estimators can be serialized and restored alongside environment metadata. The model registry additionally provides an adapter for the bio-inspired memory architecture via `create_model("fly_memory")`, supporting prediction, verified feedback updating, and state serialization.

Built-in retrieval and model composition. The RAG interface chunks packaged skills or external documents and retrieves passages using BM25 and neural rerankers. Passages retain document identifiers, source offsets, and availability timestamps;

point-in-time retrieval excludes documents unavailable at the requested decision cutoff. Citation checks validate references against retrieved passage identifiers. The pipeline freezes candidates, temporal cutoffs, and context budgets prior to neural reranking. On the temporal retrieval benchmark (`RAG_JEV_RESULTS.json`), point-in-time `as_of` filtering records 0 future-document leaks while lifting `Recall@3` / `nDCG@3` from 50.0% / 35.5% (fixed skill headers) to 100.0% / 100.0% under two-stage BM25 + semantic reranking. Across all 129 skills (2052 chunks) on 60 evaluation tasks, two-stage progressive manifest routing achieves 100.0% skill recall with 0.0% code-vs.-guidance fragmentation and 98.3% `Pass@3` at 4,039 mean tokens (−98.9% vs. full-library concatenation), compared to 55.0% recall and 25.0% fragmentation (75.0% `Pass@3`) for fixed-chunk BM25 (`top_k=5`).

Information availability and point-in-time processing. News and disclosure collection supports RSS/Atom feeds, static pages, GDELT, and official regulatory sources. Collection records distinguish publication time from observation time and retain revision histories. Point-in-time digests filter future, stale, and undated records while deduplicating canonical URLs.

Model choice and rule-based advice. An agent can inspect skills, discover candidate algorithms, execute chosen methods with custom parameters, and incorporate numerical outputs into its decision logic. Market-state categorization provides heuristic regime indicators (trend, range, stress) that agents can inspect as additional evidence alongside price tables.

Research workflow and audit boundaries. Research interfaces enforce chronological validation splits, out-of-sample holdout reporting, and standardized trial records. Python and JSON tool interfaces return structured dictionary outputs, ensuring that all intermediate calculations can be logged and audited.